

Guia de Estudio — Tema 09.2: Aliases, Closures, GC y Tipos

Materia: Paradigmas y Lenguajes de Programacion — IF009 **Institucion:** Universidad Nacional de Tierra del Fuego — Instituto IDEI **Ciclo lectivo:** 2026 (1er semestre) **Modulo:** VI — Entidades y Ligaduras (segunda parte) **Semana:** 9.2 — segunda clase del bloque Variables/Binding **Tema:** 09.2 — Aliases, Closures, GC y Tipos **Duracion de la clase:** 120 min (1 clase) **Lenguaje principal:** TypeScript **Lenguajes de contraste:** Python, Kotlin, Go, Rust, Haskell, Scala, C (referencia historica) **Fuente bibliografica principal:** Sebesta, *Concepts of Programming Languages* (Pearson, 2019), Caps. 5, 6, 10 **Fuentes secundarias:** Gabbrielli & Martini (Springer, 2023), Caps. 7, 8, 11, 16; Louden & Lambert (Cengage, 2012), Caps. 7, 9, 10 **Agente:** Dra. Sofia — Study Guide Writer **Fecha:** 2026-06-28

1. Introduccion al tema

Esta guia cubre la segunda parte del bloque **Variables, Binding y Ambito**. En la clase 09.1 formalizamos la variable como una 5-tupla (nombre, direccion, tipo, valor, lifetime), las cuatro categorias de variables (static, stack-dynamic, heap-dynamic explicita, heap-dynamic implicita) y el ambito estatico. Hoy respondemos cuatro preguntas que quedaron abiertas:

1. **Que pasa cuando dos nombres apuntan al mismo objeto?** (Aliases)
2. **Como puede una funcion acceder a variables que ya “murieron”?** (Closures)
3. **Quien libera la memoria del heap y cuando?** (Garbage Collection)
4. **Como conviven el tipado estatico y el dinamico en un mismo lenguaje?** (Gradual Typing)

Ademas cerramos con un bloque dedicado a **programacion funcional**: por que en Haskell no existen las variables mutables y como TypeScript adopta un estilo funcional con `const`, `Readonly<T>` y `reduce`.

Por que importa este tema academicamente? Porque los aliases, las closures y el GC son los tres mecanismos que determinan **el ciclo de vida real** de las variables en lenguajes modernos. Sin entenderlos, no se puede razonar sobre la memoria de un programa TypeScript, Python, Go o Rust. Y porque el gradual typing de TypeScript es el caso de estudio mas influyente del diseno de lenguajes de la ultima decada [Gabbrielli & Martini, §16.9].

Como usar esta guia: lee las secciones 2 y 3 para ubicarte. Luego recorre el desarrollo teorico (seccion 4) en orden — cada bloque referencia la filmina correspondiente con `[F-XX]`. Resuelve los ejemplos trabajados (seccion 5) con lapiz y papel antes de mirar la solucion. Al final, usa la autoevaluacion (seccion 7) para verificar que puedes explicar cada concepto con tus propias palabras.

2. Objetivos de aprendizaje

Al terminar esta guía debes poder:

#	Objetivo	Nivel Bloom
OA1	Analizar alias: identificar sus fuentes (referencias de objeto, parametros ref, uniones), razonar sobre sus implicancias en verificacion formal y analisis estatico	Analizar
OA2	Analizar closures: comparar binding profundo vs. superficial, evaluar consecuencias en el ciclo de vida de variables y semantica de programas funcionales	Analizar
OA3	Comparar garbage collection (reference counting vs. mark-sweep) con gestion manual y por ownership	Analizar
OA4	Explicar gradual typing y el rol de TypeScript como lenguaje gradualmente tipado	Comprender
OA5	Contrastar variables mutables (imperativo) con bindings inmutables (funcional) usando Haskell y TypeScript funcional	Analizar
OA6	Aplicar type narrowing en TypeScript para manejar union types de forma segura	Aplicar
OA7	Detectar errores de alias y mutabilidad en codigo generado por IA; proponer correcciones	Evaluar

Estos objetivos corresponden a los 7 OA del diseno de la clase y se cubren en los bloques 1 a 7 del desarrollo teorico.

3. Conceptos previos necesarios

Esta guía asume que dominas el **Tema 09.1 — Variables, Binding y Ambito**. Si no lo viste, revisa esos apuntes antes de continuar. En particular necesitas manejar:

- **La 5-tupla de una variable:** (nombre, direccion/L-value, tipo, valor/R-value, lifetime). Hoy usaremos L-value y R-value constantemente.
- **Las 4 categorias de variables** (Sebesta §5.4.3):
 - Categoria 1 — **static:** asignadas en compilacion, lifetime = toda la ejecucion.
 - Categoria 2 — **stack-dynamic:** nacen al llamar a la funcion, mueren al retornar (activation record).
 - Categoria 3 — **heap-dynamic implicita:** el runtime decide el momento de allocacion (closures, escape analysis).
 - Categoria 4 — **heap-dynamic explicita:** el programador pide memoria con `new` / `malloc`.
- **Ambito estatico (lexical scope):** el alcance de un nombre se determina por su posicion en el codigo fuente, no por el flujo de llamadas en runtime.
- **Binding:** el momento en que un nombre se asocia a un atributo (direccion, tipo, valor). El binding de valor puede ser estatico (constantes) o dinamico (asignacion).

Repaso rapido: si te confunden los terminos “L-value” y “R-value”, piensa que el L-value es la **direccion** de la celda (donde vive la variable) y el R-value es el **contenido** de esa celda (lo que vale). La asignacion `x = 5` lee el L-value de `x` y escribe `5` en esa direccion. Un alias es tener **dos nombres con el mismo L-value**.

4. Desarrollo teorico

Bloque 1 — Aliases: dos nombres, un objeto (15 min)

4.1.1 Que es un alias

Definicion (Sebesta §5.3.3, Louden §7.7): Un **alias** ocurre cuando dos o mas nombres distintos estan vinculados a la **misma celda de memoria** en el mismo momento de la ejecucion. [F-01]

La palabra clave es **misma celda**. Un alias no es una copia: una copia crea una celda nueva con el mismo contenido (dos L-values distintos), mientras que un alias no crea nada nuevo — solo agrega un segundo nombre que apunta al mismo L-value.

ChromaDB confirma la definicion de Sebesta:

“Because there can be any number of aliases in a program, this can be very difficult in practice. Aliasing also makes program verification more difficult.” — Sebesta, *Concepts of Programming Languages* (2019), Cap. 5 §5.3.3

Y Gabbrielli lo formaliza:

“Such case of aliasing cannot be described using a simple function State: Names -> Values because with a simple function it is not possible to express the fact that a modification of the value associated with (the variable denoted by) X also reflects on the value associated with Y.”
— Gabbrielli & Martini, *Programming Languages: Principles and Paradigms* (2023), §8

Es decir: si el estado fuera una funcion simple `Nombres -> Valores`, no podria expresar que modificar `X` tambien cambia `Y`. Se necesita un nivel de indireccion adicional (la celda compartida).

4.1.2 Fuentes de alias

Hay tres fuentes principales en lenguajes modernos:

Fuente 1 — Asignacion de referencias de objeto (la mas comun en TypeScript, Python, Kotlin):

```
const obj1 = { valor: 42, nombre: "config" };
const obj2 = obj1;           // NO se crea una copia - obj2 apunta a la misma celda

obj2.valor = 99;
console.log(obj1.valor);     // 99 - obj1 fue modificado sin tocarlo directamente

console.log(obj1 === obj2);  // true - misma identidad de objeto
```

[F-02]

Observa el contraste con los primitivos: `let y = x` sobre un numero **si copia** el valor — `x` e `y` son L-values independientes. Los primitivos no generan alias por asignacion; los objetos si.

Fuente 2 — Parametros por referencia (Kotlin, Go, C++):

```
// Kotlin - el parametro 'p' recibe la misma referencia que 'origen'
data class Punto(var x: Int, var y: Int)

fun desplazar(p: Punto, dx: Int) {
    p.x += dx // modifica el objeto ORIGINAL a traves del alias p
}

val origen = Punto(0, 0)
desplazar(origen, 5)
println(origen.x) // 5 - origen fue modificado dentro de desplazar
```

[F-03]

```
// Go - punteros explicitos: el alias es visible en la firma
func duplicar(p *int) {
    *p *= 2
}

x := 10
duplicar(&x) // &x: se pasa la direccion - p es alias de x
fmt.Println(x) // 20
```

[F-03]

Fuente 3 — Los tres escenarios de Sebesta §9.5 (paso por referencia):

Sebesta identifica tres categorías de alias involuntarios que surgen del paso por referencia, todos peligrosos [F-04b]:

1. **Colision entre parametros actuales:** si dos parametros por referencia se invocan con el mismo argumento (`fun(a, a)`), ambos nombres internos son alias del mismo objeto.
2. **Parametro y variable global:** si el argumento pasado por referencia coincide con una variable global accesible dentro de la funcion, hay dos caminos al mismo dato.
3. **Elemento de array y array completo:** `fun1(list[i], list)` — el primer parametro es alias de `list[i]`, el segundo da acceso a todo el array.

“Otro problema del pasaje por referencia es que pueden crearse alias. [...] Los problemas con estos tipos de aliasing son los mismos que en otras situaciones de aliasing: hacen que los programas sean dificiles de leer y mantener.” — Sebesta §9.5

Sebesta §9.5.2.4 propone **pass-by-value-result** como alternativa que elimina estos tres escenarios: el parametro recibe una copia al inicio y la escribe de vuelta al final — sin alias en ningun momento.

4.1.3 Consecuencias de los alias

Los alias introducen **dependencias invisibles** entre partes del programa [F-04]:

- **Razonabilidad local:** una funcion que recibe dos parametros puede asumir que son independientes. Si son alias del mismo objeto, ese supuesto falla silenciosamente. Ejemplo: `f(a, b)` donde `a.valor = 10; b.valor = 20; return a.valor + b.valor` devuelve 40 (no 30) si `a` y `b` son alias.
- **Verificacion formal:** las precondiciones de la logica de Hoare no se pueden establecer correctamente si hay alias. El analisis estatico no puede probar ausencia de efectos laterales.
- **Optimizacion del compilador:** no puede reordenar lecturas si hay alias potenciales — no sabe si dos variables se solapan.
- **Concurrencia:** dos threads con alias al mismo objeto necesitan sincronizacion explicita. Sin ella: **race condition**.

4.1.4 Guardrails: `readonly`, shallow copy y deep copy

TypeScript ofrece tres herramientas para defenderse de alias peligrosos:

`readonly` — impide la mutacion a traves del alias (no impide la existencia del alias) [F-05]:

```
function procesarBien(data: readonly number[]): number[] {  
  // data.push(99);  
  // Error: Property 'push' does not exist on type 'readonly number[]'  
  return [...data, 99]; // crea un nuevo array – el original queda intacto  
}
```

Shallow copy con spread `{...obj}` — copia solo el primer nivel [F-06]:

```
const original = { nombre: "Ana", config: { debug: true } };
const copia = { ...original };

copia.nombre = "Carlos";
console.log(original.nombre); // "Ana" — nivel 0 independiente

copia.config.debug = false;
console.log(original.config.debug); // false — nivel 1 sigue siendo alias
console.log(original.config === copia.config); // true — mismo L-value
```

Deep copy con `structuredClone` — copia completa en todos los niveles (ES2022) [F-07]:

```
const copia = structuredClone(original);
copia.config.debug = false;
console.log(original.config.debug); // true — sin alias
console.log(original.config === copia.config); // false — L-values distintos
```

Regla practica: `readonly` protege contra mutacion involuntaria. `{...obj}` protege el primer nivel. `structuredClone` protege todos los niveles. Elige segun la profundidad del objeto.

Bloque 2 — Closures: el entorno que viaja con la funcion (20 min)

4.2.1 Que es una closure

Definicion (Sebesta §10, Gabbrielli §7.4): Una **closure** es la combinacion de (1) una **funcion** (su codigo ejecutable) y (2) el **entorno lexico** en el que fue definida — el conjunto de nombres y sus valores visibles en ese punto del codigo fuente. [F-08]

Sin closures, una funcion solo puede acceder a sus parametros, sus variables locales y las variables globales. Con closures, ademas puede acceder a las **variables del scope exterior** aunque ese scope haya cerrado (de ahi el nombre “closure” — cierre).

La definicion formal de Gabbrielli:

“Clausuras: las estructuras de datos compuestas por un fragmento de codigo y un entorno de evaluacion, denominadas clausuras, constituyen el modelo canonico para implementar la llamada por nombre y todas aquellas situaciones en que una funcion debe pasarse como parametro o retornarse como resultado.” — Gabbrielli & Martini §7.4

4.2.2 El ciclo de vida extendido — migracion al heap

La pregunta clave es: **como puede una closure acceder a variables cuyo scope ya termino?** [F-10b]

Gabbrielli plantea el conflicto en §7.4:

“Cuando el resultado de $F()$ se asigna a gg , la clausura que forma su valor apunta a un entorno que contiene el nombre x . Pero este entorno es local a F y sera destruido al terminar su ejecucion. Como es posible, entonces, invocar gg posteriormente sin producir una referencia colgante a x ?” — Gabbrielli & Martini §7.4

En C, retornar un puntero a una variable local produce exactamente eso: un **dangling reference** — apunta a stack memory ya reciclada. Es un bug. La solución en lenguajes modernos:

1. El compilador detecta que la variable **escapa** del scope (es capturada por una closure).
2. La variable se aloja en el **heap** desde el inicio — no en el stack frame.
3. El closure object contiene una referencia directa a esa celda heap.
4. Cuando la función retorna y su stack frame se destruye, la variable en heap **sigue viva**.
5. El GC la libera solo cuando ninguna closure la referencia.

Esto conecta directamente con la **Categoría 3** (heap-dynamic implícita) del Tema 09.1.

Ejemplo en TypeScript [F-09]:

```
function crearContador(inicio: number) {
  let cuenta = inicio; // capturada — migra al heap

  return {
    incrementar: () => ++cuenta, // closure 1: captura cuenta
    decrementar: () => --cuenta, // closure 2: captura la MISMA cuenta
    valor:      () => cuenta    // closure 3: solo lectura
  };
}

const c = crearContador(10);
// crearContador() ya retorna — su activation record fue destruido
// pero cuenta sigue viva en el heap porque c la referencia

console.log(c.incrementar()); // 11
console.log(c.incrementar()); // 12
console.log(c.decrementar()); // 11
console.log(c.valor());       // 11
```

Las tres closures **comparten la misma celda heap** de `cuenta` — son alias controlados y encapsulados.

4.2.3 Closures en Python, Go y Kotlin

Python requiere `nonlocal` para escribir en la variable capturada [F-11]:

```
def crear_acumulador(inicio: int):
    total = inicio

    def agregar(n: int) -> int:
        nonlocal total # REQUERIDO para ESCRIBIR
        total += n
        return total

    def obtener() -> int:
        return total # LECTURA: no necesita nonlocal

    return agregar, obtener

agregar, obtener = crear_acumulador(0)
print(agregar(5)) # 5
print(agregar(3)) # 8
print(obtener())  # 8
```

Go y Kotlin soportan closures de primera clase con la misma semántica de captura [F-12]:

```
func crearContador(inicio int) func() int {
    cuenta := inicio
    return func() int {
        cuenta++
        return cuenta
    }
}
```

```
fun crearContador(inicio: Int): () -> Int {
    var cuenta = inicio
    return { ++cuenta }
}
```

C no tiene closures verdaderas [F-13]: los function pointers solo contienen la direccion del codigo, no del entorno. Toda variable no-local debe ser global. Esto ilustra por contraste cuanto valor aporta el runtime de un lenguaje moderno.

4.2.4 Deep binding vs. shallow binding

Definicion (Gabbrielli §7.4): La **politica de binding** define el momento en que se captura el entorno. [F-14]

- **Deep binding (vinculacion profunda):** el entorno se captura **al crear** la closure. Los nombres y sus valores “se congelan” en ese instante. Comportamiento predecible e intuitivo. Todos los lenguajes modernos usan deep binding: TypeScript, Python, Go, Kotlin, Haskell, Rust.
- **Shallow binding (vinculacion superficial):** el entorno se resuelve **al llamar** a la funcion, no al crearla. La misma closure puede dar resultados distintos segun donde se la invoque. LISP clasico (pre-Scheme) usaba shallow binding con ambito dinamico. Practicamente abandonado.

ChromaDB confirma:

“All common languages that use static scope also use deep binding.” — Gabbrielli & Martini §7.4

“Under static scope and deep binding, the call h(3) returns 4 (and g returns 6). The x in the body of f when it is called using h is the one in the outermost block.” — Gabbrielli & Martini §7.4

“Under dynamic scope and shallow binding, the call h(3) returns 5 (and g returns 7). The x in the body of f at the moment of its call through h is the one local to g.” — Gabbrielli & Martini §7.4

4.2.5 El ejemplo canonico de Sebesta: `makeAdder`

Sebesta §10.6.4 presenta el ejemplo canonico de closures independientes [F-14b]:


```
const makeAdder = (x: number) => (n: number): number => x + n;

const add10 = makeAdder(10); // closure 1: x = 10 en heap – celda A
const add5 = makeAdder(5); // closure 2: x = 5 en heap – celda B (independiente)

console.log(add10(1)); // 11 – celda A: x=10, n=1
console.log(add5(7)); // 12 – celda B: x=5, n=7
console.log(add10(10)); // 20 – celda A sigue siendo x=10
console.log(add5(5)); // 10 – celda B sigue siendo x=5
```

“La variable `x` referenciada en la función clausura está ligada al parámetro enviado a `makeAdder`. La función `makeAdder` se invoca dos veces: una con el parámetro 10 y otra con el parámetro 5, produciendo dos clausuras diferentes.” — Sebesta §10.6.4

Cada llamada a `makeAdder` crea su propio activation record con su propio `x`. Las dos clausuras son **completamente independientes** — no son alias entre sí. Esto contrasta con `crearContador` (F-09), donde las tres clausuras retornadas **comparten** la misma `cuenta`.

	<code>makeAdder</code>	<code>crearContador</code>
Fuente del valor capturado	Parámetro de la función	Variable local de la función
Compartido entre clausuras?	No — celda nueva por llamada	Sí — las tres clausuras comparten <code>cuenta</code>
Mutabilidad del capturado	Inmutable (solo lectura)	Mutable (lectura y escritura)
Uso típico	Currificación, partial application	Estado encapsulado, módulo con estado

4.2.6 El bug clásico de `var` en loops

El bug más famoso de clausuras en JavaScript/TypeScript [F-15, F-16]:

```
// var es función-escopo: existe UNA SOLA variable i para todo el loop
const funcs: (() => number)[] = [];

for (var i = 0; i < 3; i++) {
  funcs.push(() => i); // TODAS las clausuras apuntan a la MISMA i
}

console.log(funcs[0]()); // 3 – esperábamos 0
console.log(funcs[1]()); // 3 – esperábamos 1
console.log(funcs[2]()); // 3 – esperábamos 2
```

El deep binding **no es el bug** — la captura es correcta. El bug es que `var` comparte la **misma celda** entre todas las iteraciones. Cuando el loop termina, `i === 3`, y todas las clausuras leen esa misma celda.

Corrección con `let` (block-scope: nueva variable por iteración):

```
const funcs2: (() => number)[] = [];

for (let j = 0; j < 3; j++) {
  funcs2.push(() => j); // cada closure captura su PROPIA j
}

console.log(funcs2[0]()); // 0
console.log(funcs2[1]()); // 1
console.log(funcs2[2]()); // 2
```

Regla practica: en TypeScript moderno, **nunca uses** `var` . `let` y `const` tienen semantica de scope predecible.

Bloque 3 — Garbage Collection: memoria automatica (22 min)

4.3.1 El problema del heap

En el stack, el “cuando liberar” esta determinado por el control de flujo: cuando la funcion retorna, el frame se destruye. En el heap **no hay frame** — las celdas se crean con `new / malloc` y permanecen hasta que algo las libere [F-17].

El dilema fundamental:

- **Liberar demasiado pronto** → **dangling pointer**: una referencia apunta a memoria ya liberada. Crash o corrupcion silenciosa.
- **Liberar demasiado tarde** → **memory leak**: la memoria nunca se recupera. El proceso crece hasta agotar la RAM.
- **No liberar manualmente** → necesitamos un mecanismo automatico: el **Garbage Collector**.

Sebesta define el dangling pointer en §6.11:

“A dangling pointer, or dangling reference, is a pointer that contains the address of a heap-dynamic variable that has been deallocated. Dangling pointers are dangerous for several reasons. First, the location being pointed to may have been reallocated to some new heap-dynamic variable.” — Sebesta, *Concepts of Programming Languages* (2019), §6.11

Y senala la solucion:

“The best solution to the dangling-pointer problem is to take deallocation of heap-dynamic variables out of the hands of programmers. If programs cannot explicitly deallocate heap-dynamic variables, there will be no dangling pointers.” — Sebesta §6.11

4.3.2 Cuatro estrategias de gestion de memoria

[F-18]

Estrategia	Quien libera	Cuando libera	Ejemplos	Problema principal
Manual	El programador (<code>free</code> , <code>delete</code>)	Cuando el programador lo decide	C, C++	Dangling pointers, double-free, memory leaks
Reference Counting	El runtime (contador por celda)	Cuando el contador cae a 0	Python, Swift, PHP	No resuelve ciclos de referencia
Mark-and-Sweep	El GC	Cuando el allocator necesita espacio	Java, JavaScript/V8, Go	Pausa el programa (stop-the-world)
Ownership + Drop	El compilador (análisis estático)	Al salir del scope — garantía en compilación	Rust	Modelo de programación más restrictivo

4.3.3 Reference Counting (RC)

Cada celda del heap mantiene un contador de cuantas referencias activas la apuntan [F-19]:

- Cuando se crea una nueva referencia → `ref_count += 1`
- Cuando una referencia se destruye o reasigna → `ref_count -= 1`
- Cuando `ref_count == 0` → la celda es inaccesible → **liberar inmediatamente**

Ventajas (Sebesta §6.11.7.1) [F-19b]:

1. **Sin stop-the-world:** no hay ciclo de GC que pause el programa.
2. **Determinístico:** el momento de liberación es predecible — útil para recursos con destructores.
3. **Local:** la decisión la toma cada celda por su propio contador.

El problema crítico: referencias circulares (Louden §10.5):

“However, the overhead to maintain reference counts is not the worst flaw of this scheme. Even more serious is that circular references can cause unreferenced memory to never be deallocated.” — Louden & Lambert, *Programming Languages: Principles and Practices* (2012), §10.5

Si dos objetos se apuntan mutuamente, sus contadores nunca llegan a 0 aunque el programa no pueda alcanzarlos — son inaccesibles pero el RC no los libera. Es una **fuga de memoria estructural** del algoritmo [F-21].

Python usa RC + **cycle detector** para resolver esto. Swift usa ARC + referencias `weak` que el programador declara manualmente. Rust prohíbe los ciclos mutables en compilación.

Ejemplo en Python — observando el contador [F-20]:

```
import sys

lista = [1, 2, 3]
print(sys.getrefcount(lista)) # 2 - getrefcount crea una ref temporal

alias = lista
print(sys.getrefcount(lista)) # 3

del alias
print(sys.getrefcount(lista)) # 2

del lista
# ref_count → 0 → liberado INMEDIATAMENTE
```

4.3.4 Mark-and-Sweep

Opera en dos fases cuando el allocator se queda sin espacio [F-23]:

1. **Mark (Marcar):** a partir de todas las **raíces** (stack, variables globales, registros de CPU), trazar transitivamente todos los objetos alcanzables → marcarlos.
2. **Sweep (Barrer):** recorrer todo el heap; las celdas **no marcadas** son inaccesibles → liberar.

“The original mark-sweep process of garbage collection operates as follows: The run-time system allocates storage cells as requested and disconnects pointers from cells as necessary, without regard for storage reclamation (allowing garbage to accumulate), until it has allocated all available cells.” — Sebesta §6.11

Ventaja clave: un ciclo entre A y B que no sea alcanzable desde ninguna raíz queda sin marcar en ambos objetos → ambos se liberan en el sweep. El problema de los ciclos desaparece.

Los tres defectos del mark-and-sweep (Gabbrielli §8.11) [F-23b]:

“The mark and sweep technique suffers from three main defects. In the first place, and this is also true for reference counting, it is asymptotically the cause of external fragmentation: live and no longer live objects are arbitrarily mixed in the heap which can make allocating a large block impossible even though the total free space is sufficient.” — Gabbrielli & Martini §8.11

1. **Fragmentacion externa:** los objetos vivos y los inaccesibles quedan entremezclados. La suma de huecos puede ser suficiente pero no hay bloque contiguo.
2. **Stop-the-world:** el algoritmo clasico pausa el programa durante las dos fases. En heaps grandes: pausas perceptibles.
3. **Actualizacion de punteros tras compactacion:** si se anade compactacion, todos los punteros a objetos movidos deben actualizarse.

La solucion a la fragmentacion: compactacion (Gabbrielli §8.11):

“To avoid the fragmentation caused by the mark-and-sweep technique, one can modify the sweeping phase turning it into a compaction phase. Live objects are moved so that they are contiguous, thus leaving a single contiguous block of free memory.” — Gabbrielli & Martini §8.11

4.3.5 GC generacional en V8

V8 (motor de TypeScript/JavaScript/Node.js) usa **GC generacional** [F-25, F-26]:

- **Hipotesis generacional:** la mayoría de los objetos mueren jóvenes (resultados de `map` / `filter`, objetos temporales, closures de un solo uso).
- **New Space (generación joven):** objetos recién creados. GC muy frecuente (Scavenger con semi-space copying), sobre un espacio pequeño → muy rápido (<1ms).
- **Old Space (generación vieja):** objetos que sobrevivieron varios ciclos. GC infrecuente (Mark-Compact incremental + concurrent), más costoso pero ocurre raramente.
- **Promoción:** un objeto que sobrevive varios ciclos en New Space se promueve a Old Space.

Sebesta resume la oposición entre RC y mark-sweep:

“These two approaches to garbage collection are, in many ways, opposite processes.” —
Sebesta §6.11.7

Algoritmo	Fortaleza	Debilidad
Reference Counting	Incremental, determinístico, sin stop-the-world	No resuelve ciclos; overhead por operación
Mark-and-Sweep	Resuelve ciclos; sin overhead por operación	Stop-the-world; no incremental (clásico)

Los GC modernos (Python, Swift, V8) combinan ambos o hibridan técnicas.

Bloque 4 — Gradual typing: TypeScript como caso paradigmático (16 min)

4.4.1 El espectro del tipado

El tipado no es una dicotomía binaria — existe un espectro continuo [F-27]:

- **Estatico puro:** los tipos se verifican en **compilación**. Si hay error, no compila. Ejemplos: Java, C, Haskell, Rust.
- **Dinamico puro:** los tipos se verifican en **ejecución**. El programa compila siempre; los errores son excepciones en runtime. Ejemplos: Python, Ruby, JavaScript.
- **Gradual:** permite mezclar — algunas partes tienen tipos estáticos, otras usan `any`. El programador elige la cobertura. Ejemplos: TypeScript, Dart, Python con `mypy`.

4.4.2 La motivación histórica

Gabbrielli §16.9 explica por qué surgió el gradual typing [F-27b]:

“A medida que el numero de proyectos de software de gran escala desarrollados con lenguajes de tipado dinamico crecio con el tiempo, los usuarios advirtieron que sacrificar las verificaciones estaticas en favor de la rapidez de prototipado era un trato desfavorable. En el tipado estatico, un tipo puede verse como un contrato que tanto el proveedor como el usuario del codigo deben respetar.” — Gabbrielli & Martini §16.9

“En el tipado gradual, los usuarios pueden modular la cantidad de informacion de tipos que proporcionan en sus programas, indicando que elementos deben verificarse estaticamente y cuales en tiempo de ejecucion.” — Gabbrielli & Martini §16.9

La palabra clave es **modular**: el programador decide que partes del codigo tienen garantias estaticas y que partes quedan dinamicas.

La base formal es de **Siek & Taha (2006)** — “Gradual Typing for Functional Languages”: introdujeron un tipo especial `?` (dinamico) compatible con cualquier tipo, la relacion de **consistencia de tipos** (`~`), y **cast implicito automatico** en los limites entre codigo tipado y no tipado.

4.4.3 TypeScript en tres niveles

Nivel 0 — `any` **implicito** (compatible con JavaScript puro) [F-29]:

```
function sumar(a, b) {  
  return a + b; // sin verificacion - puede ser numero, string, lo que sea  
}  
sumar("1", 2); // "12" - concatenacion - sin error en compilacion
```

Nivel 1 — **Tipos parciales**:

```
function sumar(a: number, b): number { return a + b; }
```

Nivel 2 — **Tipos completos + strict**:

```
function sumar(a: number, b: number): number { return a + b; }  
// sumar("hola", 3) → Error en compilacion
```

Con `strict: true` en `tsconfig.json`, TypeScript pasa de gradual a completamente estatico.

Nota de diseno (Gabbrielli §16.9): TypeScript es **deliberadamente incompleto** — acepta ciertos programas con potenciales errores de tipo por razones de usabilidad y compatibilidad con JavaScript. No garantiza correccion completa del sistema de tipos. Es una decision de diseño documentada.

4.4.4 Type Narrowing

Type narrowing es el proceso por el cual TypeScript **estrecha** el tipo de una variable dentro de cada rama del control de flujo [F-30, F-31]:

```

type Resultado = string | number | null;

function formatear(r: Resultado): string {
  if (r === null) {
    return "-"; // aqui TypeScript sabe: r es null
  }
  if (typeof r === "number") {
    return r.toFixed(2); // aqui TypeScript sabe: r es number
  }
  return r.toUpperCase(); // aqui TypeScript sabe: r es string (unico restante)
}

```

Los guardas de tipo mas comunes:

- `typeof x === "string"` → en esa rama, x es `string`
- `x === null` → en esa rama, x es `null`
- `x instanceof Clase` → en esa rama, x es `Clase`
- `"propiedad" in x` → x tiene esa propiedad
- Narrowing exhaustivo con `switch` + `default: never` → el compilador detecta casos faltantes

Narrowing exhaustivo con `never` [F-32]:

```

type Forma = "circulo" | "cuadrado" | "triangulo";

function area(f: Forma, lado: number): number {
  switch (f) {
    case "circulo": return Math.PI * lado ** 2;
    case "cuadrado": return lado ** 2;
    case "triangulo": return (Math.sqrt(3) / 4) * lado ** 2;
    default:
      const _exhaustive: never = f;
      // Si se agrega "rectangulo" al tipo y se olvida el case:
      // Error: Type '"rectangulo"' is not assignable to type 'never'
      throw new Error(`Forma no manejada: ${String(_exhaustive)}`);
  }
}

```

ChromaDB confirma la conexion con union types:

“For every variable of union type, the abstract machine maintains a hidden type tag which is implicitly set when an assignment occurs. The crucial point is that a union can be used only through a ‘conformity clause’ (a case) which specifies what to do with this variable in all cases.”
— Gabbrielli & Martini §8.4.3

Bloque 5 — Variables en programacion funcional (14 min)

4.5.1 El contraste fundamental

(Sebesta §5.8, Gabbrielli §11): En los LP imperativos, las variables son celdas de memoria mutables con L-value y R-value. En los LP funcionales puros, **no existen variables mutables** — solo bindings inmutables. [F-34]

Gabbrielli lo formaliza:

“In pure functional languages, there is neither a state nor a modifiable variable. The computation proceeds — at least in principle — by rewriting expressions, by changes that take place only in the environment and do not involve the concept of memory. If there are no modifiable variables, there is no longer the concept of memory cell that can be modified.” — Gabbrielli & Martini §11

En FP puro, computar no significa “modificar el estado de celdas de memoria”. Significa **reescribir expresiones** hasta obtener un valor.

4.5.2 Transparencia referencial

Definicion (Sebesta §7.4, Louden §9.1): Un programa tiene la propiedad de **transparencia referencial** si dos expresiones cualesquiera con el mismo valor pueden sustituirse mutuamente en cualquier punto del programa sin afectar su comportamiento. [F-34b]

Sebesta:

“A program has the property of referential transparency if any two expressions in the program that have the same value can be substituted for one another anywhere in the program, without affecting the action of the program.” — Sebesta §7.4

Louden (definicion equivalente — la “regla de sustitucion”):

“Any two expressions in a program that have the same value can be replaced by one another in any place in the program without altering the result.” — Louden & Lambert §9.1

Consecuencias formales:

1. **Razonamiento ecuacional:** si $f(x) = 10$, entonces $f(x)$ puede sustituirse por 10 en cualquier parte.
2. **Memoizacion valida:** el compilador puede cachear $f(5) = 25$ y no recalcularlo.
3. **Reordenamiento seguro:** el compilador puede evaluar $f(a)$ y $g(b)$ en cualquier orden si ambas son puras.
4. **Pruebas aisladas:** una funcion pura se prueba completamente con sus argumentos — sin setup de estado global.

Conexion con alias: los alias sobre objetos mutables **rompen** la transparencia referencial. Si a y b son alias del mismo objeto mutable y $f(a)$ modifica ese objeto, entonces $f(b)$ despues de $f(a)$ produce un resultado diferente aunque $a === b$. La inmutabilidad elimina esta categoria de problemas **por diseno**.

4.5.3 Haskell — el binding es definitivo

```
let x = 5      -- x esta vinculado a 5 para siempre en este scope
-- x = 6      -- ILEGAL: no existe el concepto de reasignacion

-- La "iteracion" en FP no usa un contador mutable — usa recursion:
sumaLista :: [Int] -> Int
sumaLista []    = 0
sumaLista (x:xs) = x + sumaLista xs    -- x es un binding nuevo en cada llamada

-- Version con fold — sin ninguna variable mutable:
suma = foldl (+) 0 [1, 2, 3, 4, 5]    -- suma = 15
```

[F-35]

4.5.4 `val` vs. `var` — Scala y Kotlin

```
val y = 5      // val: binding inmutable — como const en TypeScript
var x = 5      // var: variable mutable — como let en TypeScript

x = 10         // valido — x es var
// y = 10     // Error: reassignment to val
```

[F-36]

```
val inmutable = 42    // binding definitivo
var mutable   = 42    // variable mutable

val lista     = listOf(1, 2, 3)    // lista inmutable
val listaMut  = mutableListOf(1, 2, 3) // lista mutable
```

4.5.5 TypeScript funcional — inmutabilidad como practica

```
// Imperativo — muta el acumulador
let suma = 0;
for (const x of [1, 2, 3]) suma += x;

// Funcional — sin mutacion, solo bindings nuevos
const sumaFuncional = [1, 2, 3].reduce((acc, x) => acc + x, 0);

// Cadena de transformaciones — cada paso produce un valor nuevo
const resultado = [1, 2, 3, 4, 5]
  .filter(x => x % 2 === 0)    // [2, 4] — nuevo array
  .map(x => x * 10)           // [20, 40] — nuevo array
  .reduce((a, b) => a + b, 0); // 60 — binding final
```

[F-37]

`Readonly<T>` y `ReadonlyArray<T>` para garantizar inmutabilidad en compilacion [F-38]:

```
type ConfigApp = Readonly<{ host: string; port: number; debug: boolean }>;
const cfg: ConfigApp = { host: "localhost", port: 3000, debug: false };
// cfg.debug = true; // Error: Cannot assign to 'debug' because it is read-only

const COLORES: ReadonlyArray<string> = ["rojo", "verde", "azul"];
// COLORES.push("amarillo"); // Error: Property 'push' does not exist
```

Por que importa para IA? Los LLMs tienden a generar codigo imperativo con mutacion porque predomina en el corpus de entrenamiento. El FP reduce bugs de aliasing y estado compartido. [F-39]

Bloque 6 — Contraste multilenguaje: gestion de memoria moderna (8 min)

4.6.1 Cuatro lenguajes, cuatro estrategias

[F-40]

Aspecto	TypeScript / V8	Python	Go	Rust
Estrategia de GC	GC generacional (Scavenger + Mark-Compact)	RC + cycle detector	GC concurrent tricolor incremental	Ownership + Drop — sin GC en runtime
Dangling pointer	Imposible (GC gestiona todo)	Imposible (GC)	Imposible (GC)	Imposible — borrow checker en compilacion
Variable no inicializada	Error en compilacion (strict)	<code>NameError</code> en runtime	Zero value automatico	Error en compilacion
Alias de objeto	Referencia implicita — invisible	Referencia implicita	Puntero explicito con <code>*</code> y <code>&</code>	Borrow controlado por el compilador
Ciclos de referencia	V8 Mark-Sweep los resuelve	Modulo <code>gc</code> cycle detector	GC concurrente los resuelve	Imposibles (mutables) — borrow checker
Pausa del programa	Minor GC <1ms / Major GC incremental	RC incremental — impacto bajo	GC concurrent <1ms objetivo	Sin GC → sin pausa

4.6.2 Rust: ownership — la idea central

Rust garantiza seguridad de memoria **sin GC en runtime** mediante analisis estatico en compilacion [F-41, F-42]:

- Cada valor tiene **un unico dueño** (owner) en cada momento.
- Cuando el dueño sale del scope, el valor se destruye automaticamente (**Drop trait** — destructores determinísticos).
- La propiedad puede **transferirse** (move): el dueño anterior ya no puede usar el valor.
- La propiedad puede **prestarse** (borrow): referencias temporales que el compilador verifica.

Las tres reglas del borrow checker:

1. Puede haber **cualquier cantidad de referencias inmutables** (`&T`) al mismo tiempo (lectura compartida segura).
2. O puede haber **exactamente una referencia mutable** (`&mut T`) — pero no simultaneamente con inmutables.
3. Las referencias no pueden **vivir mas tiempo** que el valor al que apuntan (no hay dangling references).

```
fn referencia_invalida() {
    let referencia: &String;
    {
        let s = String::from("hola");
        referencia = &s;
    }
    // s sale de scope → Drop → s destruida
    // println!("{}", referencia);
    // Error: `s` does not live long enough
    // Rust detecta en compilacion que referencia apuntaria a memoria liberada
}

let v1 = vec![1, 2, 3];
let v2 = v1; // ownership transferido (move)
// println!("{}", v1); // Error: value borrowed here after move
println!("{}", v2); // v2 es el unico dueño
```

Sin GC en runtime → sin pausas → rendimiento predecible. Sin dangling pointers → verificado por el compilador, no por pruebas en runtime.

Bloque 7 — Bloque IA: alias, closures y type narrowing (12 min)

Este bloque te prepara para detectar tres patrones de error que los LLMs (ChatGPT, Copilot, etc.) generan con frecuencia.

Patron 1: IA genera alias de objeto sin advertencia

```
// Prompt: "guarda una copia de config antes de modificarla"
// IA genera (INCORRECTO – alias):
const configBackup = config; // no es copia – es alias
configBackup.debug = true; // modifica config tambien

// Correccion 1 – shallow copy (sin objetos anidados):
const configBackup1 = { ...config };

// Correccion 2 – deep copy (con objetos anidados, ES2022):
const configBackup2 = structuredClone(config);
```

[F-43, F-44]

Señales de alerta:

1. `const backup = objeto` → alias (sin duda)
2. `const copia = { ...objeto }` → alias en sub-objetos anidados
3. `const arr = [...originalArr]` → alias en cada elemento objeto

Patron 2: IA usa `var` en loops con closures

```
// IA genera (INCORRECTO):
const funcs = [];
for (var i = 0; i < 5; i++) {
  funcs.push(() => i); // todas capturan la MISMA i
}
console.log(funcs[0]()); // 5 - esperabamos 0

// Correccion 1 - let:
const funcs2: (() => number)[] = [];
for (let j = 0; j < 5; j++) {
  funcs2.push(() => j); // cada closure captura su propia j
}
console.log(funcs2[0]()); // 0

// Correccion 2 - estilo funcional:
const funcs3 = Array.from({ length: 5 }, (_, k) => () => k);
console.log(funcs3[0]()); // 0
```

[F-45, F-46]

Patron 3: codigo sin narrowing que puede crashear

```
// IA genera (INCORRECTO - sin narrowing):
function formatear(valor: string | number): string {
  return valor.toUpperCase();
  // Error: Property 'toUpperCase' does not exist on type 'string | number'
}

// Correccion con narrowing:
function formatearSeguro(valor: string | number): string {
  if (typeof valor === "string") return valor.toUpperCase();
  return valor.toString();
}
```

[F-47]

Type narrowing es el **guardrail** que transforma en error de compilacion lo que sin strict seria un crash en runtime, potencialmente dificil de reproducir.

5. Ejemplos trabajados

Ejemplo 1: Trazar alias en TypeScript y detectar mutacion compartida

Consigna: Dado el siguiente codigo, predice el valor de `original.config.debug` y `copia.config.debug` despues de ejecutar todas las lineas. Explica por que.

```
const original = {
  nombre: "Ana",
  config: { debug: true, retries: 3 },
  tags: ["admin", "user"]
};

const copia = { ...original };

copia.nombre = "Carlos";
copia.config.debug = false;
copia.tags.push("guest");
```

Solucion paso a paso:

1. `const copia = { ...original }` hace una **shallow copy**: copia los valores del primer nivel, pero los objetos anidados (`config` , `tags`) se copian por **referencia** — son alias.
2. `copia.nombre = "Carlos"` → modifica solo el nivel 0 de `copia` . `original.nombre` sigue siendo "Ana" (nivel 0 independiente).
3. `copia.config.debug = false` → `copia.config` y `original.config` son el **mismo objeto** (alias). `original.config.debug` tambien es `false` .
4. `copia.tags.push("guest")` → `copia.tags` y `original.tags` son el **mismo array** (alias). `original.tags` ahora es `["admin", "user", "guest"]` .

Resultado:

- `original.config.debug` → `false` (modificado por alias)
- `copia.config.debug` → `false`
- `original.tags` → `["admin", "user", "guest"]` (modificado por alias)
- `original.nombre` → "Ana" (sin cambios — nivel 0 independiente)

Conclusion: el spread operator `{...obj}` **no es suficiente** cuando hay objetos anidados. Para una copia real usa `structuredClone(original)` .

Ejemplo 2: Closure con bug de `var` en loop y correccion con `let`

Consigna: El siguiente codigo genera un array de 3 funciones que deberian retornar 0, 1 y 2. Sin embargo, todas retornan 3. Explica el bug y propon dos correcciones.

```
const funcs: (() => number)[] = [];  
for (var i = 0; i < 3; i++) {  
  funcs.push(() => i);  
}  
console.log(funcs[0]()); // 3 - esperabamos 0  
console.log(funcs[1]()); // 3 - esperabamos 1  
console.log(funcs[2]()); // 3 - esperabamos 2
```

Solucion paso a paso:

1. **Que hace `var`:** `var` declara la variable en el scope de la **funcion contenedora**, no del bloque. Hay **una sola variable** `i` para todo el loop — no se crea una `i` nueva en cada iteracion.
2. **Que hace la closure:** cada `() => i` captura la **referencia** a `i` (deep binding correcto). Pero como todas las closures capturan la **misma celda** de `i`, todas leen el mismo valor.
3. **El momento de la llamada:** cuando se ejecuta `funcs[0]()`, el loop ya termino. En ese punto `i === 3` (la condicion `i < 3` fallo y el loop salio con `i = 3`). Todas las closures leen esa celda que ahora contiene 3.
4. **El bug no es el deep binding:** la captura es correcta. El bug es que `var` comparte la misma celda entre todas las iteraciones.

Correccion 1 — `let` (block-scope):

```
const funcs2: (() => number)[] = [];  
for (let j = 0; j < 3; j++) {  
  funcs2.push(() => j); // cada closure captura su PROPIA j  
}  
console.log(funcs2[0]()); // 0  
console.log(funcs2[1]()); // 1  
console.log(funcs2[2]()); // 2
```

`let` crea una variable nueva en cada iteracion del bloque. Cada closure captura una celda distinta en el heap.

Correccion 2 — estilo funcional:

```
const funcs3 = Array.from({ length: 3 }, (_, k) => () => k);  
console.log(funcs3[0]()); // 0  
console.log(funcs3[1]()); // 1  
console.log(funcs3[2]()); // 2
```

`Array.from` llama al callback con un parametro `k` nuevo en cada invocacion. Los parametros son bindings fresh — no se comparten entre llamadas.

Ejemplo 3: Analisis de dangling reference vs. GC

Consigna: Compara estos dos fragmentos. En C, el primero es un bug. En TypeScript, el segundo es seguro. Explica por que.

```
// C - BUG: dangling reference  
int* crear() {  
  int x = 42;  
  return &x; // x se destruye al retornar - puntero colgante  
}  
int* p = crear();  
printf("%d", *p); // comportamiento indefinido
```

```
// TypeScript – SEGURO: closure captura y migra al heap
function crear() {
  let x = 42;
  return () => x; // closure captura x – x migra al heap
}
const f = crear();
console.log(f()); // 42 – seguro
```

Solucion paso a paso:

1. **En C:** la variable local `x` vive en el **stack frame** de `crear`. Cuando `crear` retorna, el frame se destruye y `x` deja de existir. El puntero `p` apunta a memoria ya reciclada — un **dangling reference**. Desreferenciarlo es comportamiento indefinido: puede crashear, devolver basura, o funcionar “bien” hasta que la memoria se reutilice.
2. **En TypeScript:** el compilador detecta que `x` es **capturada por una closure** (la funcion flecha `() => x`). Esto significa que `x` **escapa** del scope de `crear` — su lifetime debe ser mayor que el del activation record. El runtime aloja `x` en el **heap** desde el inicio, no en el stack. Cuando `crear` retorna y su frame se destruye, `x` en heap **sigue viva**. La closure `f` mantiene una referencia a esa celda heap. El GC liberara `x` solo cuando `f` deja de ser accesible.
3. **La diferencia fundamental:** en C, el programador es responsable de gestionar el ciclo de vida. En TypeScript (y Python, Go, Kotlin), el **runtime detecta automaticamente** que variables escapan y las mueve al heap. Es la garantia estructural que hace imposibles los dangling references en lenguajes con GC.

“The best solution to the dangling-pointer problem is to take deallocation of heap-dynamic variables out of the hands of programmers.” — Sebesta §6.11

6. Puntos clave (cheat-sheet)

Alias — resumen rapido

- **Alias = dos nombres, una sola celda** (mismo L-value). No es copia.
- **Fuentes:** (1) asignacion de referencias de objeto, (2) parametros por referencia (3 escenarios de Sebesta §9.5), (3) union types.
- **Consecuencias:** razonabilidad local falla, verificacion formal dificultada, optimizacion del compilador limitada, race conditions en concurrencia.
- **Guardrails:** `readonly` (impide mutacion), `{...obj}` (shallow copy — nivel 0), `structuredClone` (deep copy — todos los niveles).
- **const no evita aliases:** `const obj2 = obj1` — ambas constantes referencian el mismo objeto mutable.

Closures — resumen rapido

- **Closure = funcion + entorno lexico capturado** (Gabbrielli §7.4: “pair code/environment”).
- **Ciclo de vida extendido:** las variables capturadas migran del stack al heap — viven mientras exista la closure.
- **Deep binding:** el entorno se captura al **crear** la closure (todos los lenguajes modernos).
- **Shallow binding:** el entorno se resuelve al **llamar** (ambito dinamico, abandonado).
- `makeAdder` (**Sebesta §10.6.4**): cada llamada crea una celda heap independiente — closures no compartidas.
- `crearContador` : las tres closures comparten la misma celda — aliases controlados.
- `var` vs. `let` : `var` es funcion-scope (una celda compartida); `let` es block-scope (celda nueva por iteracion). **Nunca uses** `var` .

GC — resumen rapido

- **Cuatro estrategias:** Manual (C/C++), Reference Counting (Python/Swift), Mark-and-Sweep (Java/V8/Go), Ownership+Drop (Rust).
- **RC:** incremental, deterministico, sin stop-the-world. **No resuelve ciclos** (Louden §10.5).
- **Mark-and-Sweep:** resuelve ciclos. Tres defectos (Gabbrielli §8.11): fragmentacion, stop-the-world, actualizacion de punteros. Compactacion soluciona fragmentacion.
- **V8 generacional:** New Space (Scavenger, <1ms) + Old Space (Mark-Compact, infrecuente). Hipotesis generacional: la mayoría de los objetos mueren jovenes.
- **Rust:** ownership + borrow checker = seguridad de memoria sin GC en runtime. Sin pausas, sin dangling pointers (garantia en compilacion).

Gradual typing — resumen rapido

- **Espectro:** estatico puro (Java, Haskell) ↔ dinamico puro (Python, JS) ↔ gradual (TypeScript).
- **TypeScript = caso paradigmatico** (Gabbrielli §16.9). Base formal: Siek & Taha 2006.
- **Tres niveles:** `any` implicito (nivel 0) → tipos parciales → strict completo.
- **Type narrowing:** TypeScript estrecha el tipo en cada rama del control de flujo. Guardas: `typeof` , `=== null` , `instanceof` , `in` , `switch` + `never` .
- **TypeScript es deliberadamente incompleto:** acepta ciertos programas con potenciales errores por compatibilidad con JavaScript.

FP e inmutabilidad — resumen rapido

- **FP puro:** no existen variables mutables — solo bindings inmutables (Haskell, Erlang).
- **Transparencia referencial** (Sebesta §7.4, Louden §9.1): dos expresiones con el mismo valor pueden sustituirse mutuamente sin afectar el comportamiento.
- **val vs. var** (Scala, Kotlin): `val` = binding inmutable, `var` = variable mutable.
- **TypeScript funcional:** `const` + `Readonly<T>` + `ReadonlyArray` + `reduce` / `map` / `filter`.
- **La inmutabilidad elimina los alias peligrosos por diseno:** si los objetos no se pueden mutar, los alias son inofensivos.

7. Autoevaluacion

Las siguientes 10 preguntas cubren los 7 objetivos de aprendizaje. Intenta responder sin consultar la guia. Las respuestas estan en `<details>` despues de cada pregunta.

Pregunta 1 (OA1 — Recordar): Define que es un alias y en que se diferencia de una copia.

► Respuesta

Pregunta 2 (OA1 — Analizar): Sebesta §9.5 describe tres escenarios de alias involuntarios por paso por referencia. Nombra los tres y da un ejemplo de cada uno.

► Respuesta

Pregunta 3 (OA2 — Comprender): Por que las variables capturadas por una closure no pueden vivir en el stack? Que hace el runtime para resolverlo?

► Respuesta

Pregunta 4 (OA2 — Analizar): Cual es la diferencia entre deep binding y shallow binding? Da un ejemplo de lenguaje para cada uno.

► Respuesta

Pregunta 5 (OA3 — Analizar): Por que el reference counting puro no puede liberar estructuras circulares? Como lo resuelve Python?

► Respuesta

Pregunta 6 (OA3 — Comparar): Compara las cuatro estrategias de gestion de memoria (Manual, RC, Mark-Sweep, Ownership+Drop) en una tabla con: quien libera, cuando libera, y el problema principal.

► Respuesta

Pregunta 7 (OA4 — Comprender): Que es el gradual typing y por que TypeScript es su caso paradigmatico? Que significa que TypeScript sea “deliberadamente incompleto”?

► Respuesta

Pregunta 8 (OA6 — Aplicar): Escribe una funcion `formatear` que reciba un valor de tipo `string` | `number` | `null` y retorne un string. Usa type narrowing para manejar los tres casos.

► Respuesta

Pregunta 9 (OA5 — Analizar): Define transparencia referencial (Sebesta §7.4) y explica por que los alias sobre objetos mutables la rompen.

► Respuesta

Pregunta 10 (OA7 — Evaluar): Un LLM genera el siguiente código TypeScript. Identifica los tres errores y propon correcciones.

```
const config = { host: "localhost", port: 3000, nested: { debug: true } };
const backup = config;
backup.nested.debug = false;

const funcs = [];
for (var i = 0; i < 3; i++) {
  funcs.push(() => i);
}

function procesar(valor: string | number) {
  return valor.toUpperCase();
}
```

► Respuesta

8. Glosario

Termino	Definicion
Alias	Situacion en la que dos o mas nombres distintos estan vinculados a la misma celda de memoria (mismo L-value) en el mismo momento de la ejecucion [Sebesta §5.3.3].
Closure	Combinacion de una funcion y el entorno lexico en el que fue definida. Permite acceder a variables del scope exterior aunque ese scope haya terminado [Sebesta §10, Gabbrielli §7.4].
Deep binding	Politica de binding en la que el entorno se captura al crear la closure. Los nombres y valores se congelan en ese instante. Usado por todos los lenguajes modernos con ambito estatico [Gabbrielli §7.4].
Shallow binding	Politica de binding en la que el entorno se resuelve al llamar a la funcion, no al crearla. Asociado al ambito dinamico. Practicamente abandonado [Gabbrielli §7.4].
Dangling reference	Puntero o referencia que contiene la direccion de una variable heap-dynamic que ya fue liberada. Peligroso: la memoria puede haber sido reasignada a otros datos [Sebesta §6.11].
Garbage collection (GC)	Mecanismo automatico que determina cuando una celda del heap es inaccesible y la devuelve al pool de memoria libre, sin intervencion del programador [Sebesta §6.11].
Mark-and-sweep	Algoritmo de GC en dos fases: (1) Mark — trazar transitivamente desde las raices todos los objetos alcanzables y marcarlos; (2) Sweep — recorrer el heap y liberar las celdas no marcadas. Resuelve ciclos de referencia [Sebesta §6.11, Gabbrielli §8.11].
Reference counting (RC)	Algoritmo de GC en el que cada celda mantiene un contador de referencias activas. Cuando el contador llega a 0, la celda se libera inmediatamente. Incremental y deterministico, pero no resuelve ciclos de referencia [Louden §10.5].
Ownership	Sistema de Rust en el que cada valor tiene un unico dueño en cada momento. Cuando el dueño sale del scope, el valor se destruye automaticamente (Drop). Garantiza seguridad de memoria sin GC en runtime [Rust].
Gradual typing	Sistema de tipos que permite al programador modular la cantidad de informacion de tipos que proporciona — algunas partes tienen tipos estaticos, otras usan <code>any</code> (dinamico). Base formal: Siek & Taha 2006 [Gabbrielli §16.9].
Type narrowing	Proceso por el cual TypeScript estrecha (narrows) el tipo de una variable dentro de cada rama del control de flujo, basandose en las condiciones verificadas en ramas anteriores. Guardas: <code>typeof</code> , <code>=== null</code> , <code>instanceof</code> , <code>in</code> , <code>switch</code> + <code>never</code> .
Union type	Tipo que permite que un valor sea de uno de varios tipos especificados (ej: <code>string number null</code>). Se usa de forma segura mediante type narrowing [TypeScript].
Immutable	Que no puede modificarse despues de su creacion. En FP puro (Haskell), todos los bindings son inmutables. En TypeScript, <code>const</code> + <code>Readonly<T></code> + <code>ReadonlyArray</code> aproximan inmutabilidad. La inmutabilidad elimina los aliases peligrosos por diseno.

Termino	Definicion
val / var	Palabras clave de Scala y Kotlin. <code>val</code> declara un binding inmutable (no puede reasignarse). <code>var</code> declara una variable mutable (puede reasignarse). Equivalente a <code>const</code> / <code>let</code> en TypeScript.
Escape analysis	Analisis del compilador que detecta cuando una variable “escapa” del scope de la funcion (es capturada por una closure, se retorna por referencia, o se almacena en una estructura con mayor lifetime). Las variables que escapan se alojan en el heap en lugar del stack [Go, V8].

9. Referencias y lecturas recomendadas

Fuentes primarias (verificadas en ChromaDB)

1. **Sebesta, R. W.** (2019). *Concepts of Programming Languages* (12th ed.). Pearson.

- Cap. 5 §5.3.3 — Aliases: definicion y fuentes (p. 221-258).
- Cap. 5 §5.4.3 — Categorías de variables y momentos de binding.
- Cap. 5 §5.8 — Variables en programacion funcional.
- Cap. 6 §6.11 — Pointer and Reference Types; Garbage Collection; dangling pointers (p. 259-324).
- Cap. 7 §7.4 — Transparencia referencial y efectos laterales (p. 325-388).
- Cap. 9 §9.5 — Aliases por paso por referencia: tres escenarios (p. 389-440).
- Cap. 10 §10.6.4 — Closures: ejemplo canonico `makeAdder` (p. 441-470).

2. **Gabbrielli, M. & Martini, S.** (2023). *Programming Languages: Principles and Paradigms* (2nd ed.). Springer.

- §7.4 — Closures: definicion formal, deep vs. shallow binding, dangling reference (p. 136-282).
- §8.11 — Mark-and-sweep: tres defectos y compactacion (p. 136-282).
- §8.4.3 — Tagged unions y conformity clause (p. 136-282).
- §8.8 — Type checking and inference (p. 136-282).
- Cap. 11 — Programacion funcional: bindings inmutables, transparencia referencial.
- §16.9 — Gradual typing y TypeScript (p. 533-574).

3. **Louden, K. C. & Lambert, K. A.** (2012). *Programming Languages: Principles and Practices* (3rd ed.). Course Technology.

- §7.7 — Aliases (p. 054-147).
- §9.1 — Transparencia referencial: regla de sustitucion (p. 406-447).
- §10.3 — Closures (p. 448-495).
- §10.5 — Garbage Collection: reference counting y referencias circulares (p. 448-495).

Fuentes complementarias

4. **Siek, J. G. & Taha, W.** (2006). *Gradual Typing for Functional Languages*. Scheme Workshop. — Base formal del gradual typing.
5. **TypeScript Handbook**. Narrowing, Template Literal Types.
<https://www.typescriptlang.org/docs/>
6. **Filminas UNTDF 2024**. *Cuestiones semanticas vinculadas a Variables*. (ingesta/variables.pdf en ChromaDB).

Trazabilidad bibliografica (ChromaDB)

Las citas de esta guia fueron verificadas contra la knowledge base ChromaDB del curso, que contiene los 3 libros ingestados:

Referencia	Fuente ChromaDB	Capitulo / Seccion
Aliases — definicion y fuentes	Sebesta (2019)	Cap. 5 §5.3.3 (p. 221-258)
Aliases por paso por referencia — 3 escenarios	Sebesta (2019)	Cap. 9 §9.5 (p. 389-440)
Closures — deep vs. shallow binding	Gabbrielli & Martini (2023)	§7.4 (p. 136-282)
Closures — makeAdder	Sebesta (2019)	Cap. 10 §10.6.4 (p. 441-470)
GC — mark-and-sweep	Sebesta (2019)	§6.11 (p. 259-324)
GC — reference counting, ciclos	Louden & Lambert (2012)	§10.5 (p. 448-495)
GC — tres defectos del mark-sweep + compactacion	Gabbrielli & Martini (2023)	§8.11 (p. 136-282)
Dangling pointers	Sebesta (2019); Gabbrielli & Martini (2023)	§6.11; §8.4.6
Gradual typing — TypeScript	Gabbrielli & Martini (2023)	§16.9 (p. 533-574)
Type inference	Gabbrielli & Martini (2023)	§8.8 (p. 136-282)
Tagged unions / conformity clause	Gabbrielli & Martini (2023)	§8.4.3 (p. 136-282)
Transparencia referencial	Sebesta (2019); Loudon & Lambert (2012)	§7.4 (p. 325-388); §9.1 (p. 406-447)
FP — bindings inmutables	Gabbrielli & Martini (2023)	Cap. 11 (p. 351-423)

Conexiones hacia adelante

- **Tema 10 — Tipos de Datos:** union types y discriminated unions construyen directamente sobre type narrowing.
 - **Tema 11 — Programacion Funcional:** los bindings inmutables y la transparencia referencial son el fundamento del paradigma funcional puro.
 - **Tema 14 — Sistemas de Tipos:** TypeScript como gradual typing, inferencia de tipos, strict mode en profundidad.
-

Generado por Dra. Sofia — Study Guide Writer (EDU) 1 clase x 120 min | Fuentes: Sebesta Cap. 5/6/7/9/10 + Gabbrielli Cap. 7/8/11/16 + Louden Cap. 7/9/10 ChromaDB: 12 citas verificadas | Autoevaluacion: 10 preguntas Bloom | Glosario: 15 terminos Estado: Pendiente de revision docente